

ASSIGNMENT 03 (QUIZ 5B)

Name: _____

Roll No: _____

COAL – FALL 2024

Section: _____

Question 01**(5 Marks)**

Dry run the following code and update the memory at the end of execution.

```
.data
array DWORD 0348C2367h, 01C933A79h, 06F3A4B76h
```

```
.code
    mov bl, 4
    mov esi, OFFSET array
    mov ecx, (LENGTHOF array) - 1
```

```
L1:
    push ecx
    mov eax, [esi + TYPE DWORD]
    mov cl, bl
    shld [esi],eax,cl

    add esi, TYPE DWORD
    pop ecx
    Loop L1
```

```
shl DWORD PTR [esi],4
```

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4000	71	36	C2	48	96	A7	33	C9	60	B7	A4	F3	00	00	00	00

Question 02**(5 Marks)**

Fill in the table for the following instructions and make sure to perform the calculations. Note: The hexadecimal representation of '0' is 30h.

<pre>.code main proc mov ax, 0 mov al, '5' sub al, '2' add al, '7' aaa or ax, 3030h</pre>	<table><tr><td>ax</td><td>00 00</td></tr><tr><td>al</td><td>35</td></tr><tr><td>al</td><td>03</td></tr><tr><td>al</td><td>3A</td></tr><tr><td>ax</td><td>31 30</td></tr></table>	ax	00 00	al	35	al	03	al	3A	ax	31 30	<u>CALCULATION</u>
ax	00 00											
al	35											
al	03											
al	3A											
ax	31 30											
<pre>.code main proc mov ax, 0 mov al, '5' sub al, '9' aas or ax, 3030h</pre>	<table><tr><td>ax</td><td>00 00</td></tr><tr><td>al</td><td>35</td></tr><tr><td>al</td><td>FC</td></tr><tr><td>al</td><td>06</td></tr><tr><td>ax</td><td>FF 36</td></tr></table>	ax	00 00	al	35	al	FC	al	06	ax	FF 36	<u>CALCULATION</u>
ax	00 00											
al	35											
al	FC											
al	06											
ax	FF 36											

Question 03**(5 Marks)**

Execute the following code and fill the table accordingly.

```
.data
```

```
dwordVal SDWORD -30000
```

```
.code
```

```
mov eax, dwordVal
```

```
cdq
```

```
mov ebx, 256
```

```
idiv ebx
```

	Answers	Registers (EAX/EBX/ECX/EDX)
Quotient	FF FF FF 8B	EAX
Remainder	FF FF FF D0	EDX

```
.data
```

```
wordVal SWORD 3000
```

```
.code
```

```
mov ax, wordVal
```

```
cwd
```

```
mov bx, -256
```

```
idiv bx
```

	Answers	Registers (EAX/EBX/ECX/EDX)
Quotient	FF F5	AX
Remainder	00 B8	DX

Question 04**(10 Marks)**

Write a **recursive** assembly language program to calculate the n-th term of the **Pell series**. The Pell series is defined as follows:

$$P(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ 2 \cdot P(n-1) + P(n-2) & \text{if } n > 1 \end{cases}$$

An example of the series till n = 4 is 0, 1, 2, 5, 12.

```

.386

.model flat, stdcall

.stack 4096

include Irvine32.inc


.data

n DWORD 6                ; Desired term (n) in the Pell series


.code

main PROC

    mov eax, n            ; Load the term number (n) into eax
    push eax              ; Push n as a parameter for Pell(n)
    call Pell              ; Call the Pell function
    add esp, 4             ; Clean up the stack

    ; Result is now in eax, exit
    exit

main ENDP


Pell PROC

    push ebp              ; Save base pointer
    mov ebp, esp          ; Establish stack frame
    sub esp, 4            ; Allocate local variable

    mov eax, [ebp + 8]    ; Get the input parameter (n)

```

```

    cmp eax, 0          ; Compare n with 0
    je base_case_0      ; If n == 0, go to base case 0

    cmp eax, 1          ; Compare n with 1
    je base_case_1      ; If n == 1, go to base case 1

; Recursive case: 2 * Pell(n-1) + Pell(n-2)

    push eax            ; Save n
    dec eax             ; n = n - 1
    push eax            ; Push n-1
    call Pell           ; Compute Pell(n-1)
    add esp, 4          ; Clean up the stack
    mov ebx, eax        ; Save Pell(n-1) in ebx

    pop eax             ; Restore n
    dec eax             ; n = n - 2
    push eax            ; Push n-2
    call Pell           ; Compute Pell(n-2)
    add esp, 4          ; Clean up the stack

    lea eax, [ebx + eax * 2] ; eax = 2 * Pell(n-1) + Pell(n-2)
    jmp end_pell        ; End of the function

base_case_0:
    mov eax, 0          ; Pell(0) = 0
    jmp end_pell        ; End of the function

```

```
base_case_1:
    mov eax, 1          ; Pell(1) = 1
    jmp end_pell        ; End of the function

end_pell:
    mov esp, ebp        ; Restore stack pointer
    pop ebp             ; Restore base pointer
    ret                 ; Return to the caller

Pell ENDP

END main
```